Operating Systems Project — Review 2

Methodology and Partial Implementation (Lab Assessment 9)

Event-Driven Runtime-Adaptive CPU Scheduling with Sequential Tabular Q-Learning

- **Repository:** github.com/lostpilot404/OS-Project
- **Language and libraries:** Python 3.11.2 with NumPy 2.4.6, pandas 3.0.6 and pytest 9.1.1
- **Automated verification:** 273 checks, all passing (re-run during this build)
- **Dataset:** 7 synthetic families: 6,000 training episodes, 140 validation workloads, 210 held-out test workloads
- **Date:** 12 October 2026

1. Detailed methodology of the proposed system

The proposed system is a single-CPU, event-driven, runtime-adaptive CPU scheduler. At every dispatch or Round-Robin quantum boundary, a sequential tabular Q-learning controller chooses one of four classical policies (FCFS, SJF, Round Robin, Priority) for the next execution segment of the same evolving trace. The controller sees only work that has already arrived, so every measured gain can be attributed to information a real scheduler could have had.

1.1 Research questions

- RQ1 — Can a single-CPU scheduler switch between FCFS, SJF, Round Robin and Priority at run time, using only information available at that instant?
- RQ2 — Does such switching reduce mean waiting time relative to the fixed policies it switches between, once a context-switch cost is charged?
- RQ3 — Does a learned switching policy add value over a hand-written adaptive rule that sees the same observation?
- RQ4 — Do the gains hold on a workload family never seen during training, and are they larger than the paired bootstrap noise?

1.2 Methodology pipeline

The method is a closed loop between a discrete-event environment, which owns the future, and a controller, which sees only the past and the present. Stages 3 to 7 run at every decision epoch; stage 8 runs once per experiment.

Diagram 1 — Methodology pipeline (section 1.2)

```mermaid
flowchart LR
    S1["1. Dataset<br/>7 seeded families, 15 processes"]
    S2["2. Simulation<br/>single CPU, FIFO ready queue"]
    S3["3. Observation<br/>arrived work only"]
    S4["4. Encoding<br/>5 features, 162 states"]
    S5["5. Decision<br/>Q-learning picks a policy"]
    S6["6. Execution<br/>segment runs, switch cost charged"]
    S7["7. Reward<br/>r = -dW / q"]
    S8["8. Evaluation<br/>held-out test and paired bootstrap"]
    S1 --> S2
    S2 --> S3
    S3 --> S4
    S4 --> S5
    S5 --> S6
    S6 --> S7
    S7 --> S8
    S7 -.->|"repeat at every decision epoch"| S3
```

- **Stage 1 — Dataset —** seven seeded families of 15-process workloads (workload/generator.py).
- **Stage 2 — Simulation —** one CPU, one FIFO ready queue, remaining bursts, switch cost (scheduler/runtime.py).

- **Stage 3 — Observation —** an immutable snapshot of arrived and ready work only (scheduler/runtime.py).
- **Stage 4 — Encoding —** five causal features, discretised into 162 states (3 x 3 x 3 x 3 x 2) (rl/runtime_state.py).
- **Stage 5 — Decision —** epsilon-greedy in training, greedy in evaluation, over four policy actions, with a counted fallback in unseen states (rl/runtime_controller.py).
- **Stage 6 — Execution —** run the segment (Round Robin bounded by the quantum); charge one unit when the running PID changes (scheduler/runtime.py).
- **Stage 7 — Reward —** the waiting-time area dW of ready processes gives r = -dW / q, with gamma = 1 (scheduler/runtime.py).
- **Stage 8 — Evaluation —** greedy, read-only runs on held-out workloads; paired differences with a crossed bootstrap (experiments/runtime_experiment.py).

1.3 Causal observation contract

The environment privately owns the complete event list, so it can admit future arrivals at their arrival times. The controller receives only an immutable RuntimeObservation; the encoder and controller interfaces accept no Workload object.

The observation contains:

- the current simulated time and the policy chosen at the previous epoch (None at the first epoch);
- the arrived, ready processes, with PID, arrival time, known burst, remaining burst and priority;
- counts of arrived and completed processes, the observed mean burst and priority spread of arrived work, and the mean arrival age of ready jobs (current time minus arrival time, which includes earlier service and is not accumulated queue waiting).

It deliberately excludes future arrivals and their bursts or priorities, the workload size and family, and the metrics of policies that were not run. A paired hidden-future test changes unarrived processes and asserts that the observation and action prefixes stay identical. Exact burst lengths are assumed known once a process arrives, the same assumption standalone SJF requires; no burst predictor is used.

1.4 State abstraction and learning objective

The encoder maps each observation to one of 162 states, with quantum q = 4:

- **ready_count —** 1; 2–3; 4 or more
- **completed_count —** 0; 1–3; 4 or more
- **median_ready_remaining_burst —** at most q; at most 4q; more than 4q
- **mean_ready_arrival_age —** at most q; at most 4q; more than 4q
- **arrived_work_priority_spread —** zero; non-zero

Each trace is one sequential episode. Between decision epochs the environment measures the waiting-time area dW accumulated by ready processes, including any switch interval, and returns:

```
r_t = -dW_t / max(1, q)        gamma = 1.0
sum over t of r_t = -TotalWaitingTime / q
```

Because gamma = 1 and every episode is finite, maximising the return minimises total waiting time. The tabular update is:

```
Q(s, a) <- Q(s, a) + alpha * ( r_t + gamma * max over visited a' of Q(s', a') - Q(s, a) )
alpha = 0.1; terminal transitions omit the bootstrap term; Q initialised to 0
```

Exploration: epsilon starts at 1.0, is multiplied by 0.997 after each episode and is floored at 0.05. The first visit to a state forces a uniformly random action. Greedy choices and bootstrap targets use only actions with a positive visit count. A wholly unseen state in evaluation triggers a counted fallback to the predeclared heuristic.

1.5 Methodological controls

- **Baselines —** FCFS, SJF, Round Robin and Priority run on identical workloads with the identical switch cost. A predeclared, non-learning adaptive rule is fixed before the held-out test.
- **Blinded held-out split —** the final test (master seed 19301) is generated after training and validation, and the pipeline asserts zero fingerprint overlap with both.

- **Retired seed —** an earlier final-test seed (9301) was examined during an audit and is retired; no result from it is reported.
- **Out-of-family generalisation —** poisson_arrivals is excluded from training and used only at evaluation time.
- **Independent seeds and paired uncertainty —** five agents (seeds 7101–7105) are trained separately; differences are paired per workload and resampled with a family-stratified crossed bootstrap of 1,000 replicates (seed 104729).
- **Determinism and timing —** evaluation is greedy and read-only; every artifact regenerates from the declared seeds; wall-clock timings are recorded separately and never charged to simulated time.

2. Overall conceptual, architectural and pipeline diagram

The information boundary is structural: the environment owns the future and the accounting, the controller receives a frozen snapshot of arrived work, and its reply is a single policy name. The controller cannot modify the queue, the bursts or the event list.

Diagram 2 — System architecture (section 2)

```mermaid
flowchart TB
    subgraph ENV["Environment: scheduler/runtime.py (owns the future)"]
        E1["Private event list: future arrivals"]
        E2["Single FIFO ready queue and remaining bursts"]
        E3["CPU segment, switch cost, schedule trace"]
    end
    subgraph CTL["Controller: arrived work only"]
        C1["RuntimeObservation (frozen snapshot)"]
        C2["State encoder: 162 states"]
        C3["Q-learning controller: 648 state-action pairs"]
    end
    M["Waiting-time increment gives reward r = -dW / q"]
    E1 --> E2
    E2 --> E3
    E3 -->|"observation"| C1
    C1 --> C2
    C2 --> C3
    C3 -->|"action: FCFS, SJF, RR or Priority"| E3
    E3 --> M
    M -.->|"Q update"| C3
```

- **Environment —** scheduler/runtime.py: private event list, FIFO ready queue, remaining bursts, switch cost and trace.
- **Perception and decision —** rl/runtime_state.py, rl/runtime_controller.py, rl/runtime_heuristic.py: state encoding, the tabular Q-learning policy and the predeclared rule.
- **Baselines —** scheduler/fcfs.py, sjf.py, round_robin.py, priority.py: standalone fixed policies, used to check runtime actions.
- **Experiment —** experiments/runtime_experiment.py, experiments/runtime_config.py: splits, training, validation, held-out testing, bootstrap and artifact export.
- **Data and metrics —** workload/models.py, workload/generator.py, evaluation/metrics.py: Process and Workload objects, seeded families, and metric definitions.

2.1 Runtime control flow

1. The environment advances to the next decision epoch (a dispatch point, a completion or a quantum boundary) and admits every arrival up to that instant, in (arrival time, pid) order.
2. It freezes an observation and passes it to the controller, which returns one of four policy names.
3. If the selected process differs from the running PID, one unit of switch cost is charged before the segment starts; the chosen dispatch is not reconsidered.
4. The segment runs. Round Robin is bounded by the quantum; the other policies run to completion. An unfinished Round-Robin process is appended to the tail after endpoint arrivals.

5. The waiting-time area is converted to the interval reward and returned to the controller for the update.

3. Description of the major modules

Six modules carry the system: two build and run the simulated machine (M1, M2), two implement the learned controller (M3, M4), one is the non-learning reference rule (M5), and one orchestrates the experiment and its statistics (M6).

- **M1 — Workload generator and dataset builder.** Files: workload/generator.py, workload/models.py. Contract: family configuration and seed validated workload with fingerprint. It generates the seven synthetic families and validates every process (non-negative PID, arrival and priority; positive burst; unique PIDs).
- **M2 — Discrete-event runtime environment.** Files: scheduler/runtime.py. Contract: workload, controller, quantum and switch cost schedule trace and six metrics. It owns the private event list, the single FIFO ready queue, remaining bursts, switch-cost accounting and the interval reward.
- **M3 — Causal observation and state encoder.** Files: rl/runtime_state.py. Contract: RuntimeObservation state index in [0, 162). It extracts five causal features, discretises them relative to the quantum and flattens them in mixed-radix order. It rejects any object other than an observation, including a Workload.
- **M4 — Sequential Q-learning controller.** Files: rl/runtime_controller.py, rl/q_learning.py. Contract: observation (encode) and reward (update) policy action; Q table and visit counts. Tabular Q-learning over 162 x 4 = 648 state-action pairs, with epsilon-greedy exploration, first-visit forcing, unvisited-action masking, read-only greedy evaluation and a counted heuristic fallback.
- **M5 — Predeclared heuristic controller.** Files: rl/runtime_heuristic.py. Contract: RuntimeObservation policy action. A non-learning rule (Round Robin, then Priority, then SJF, then FCFS) that uses the same observation as the learner and has no fitted parameters.
- **M6 — Experiment orchestration, metrics and reporting.** Files: experiments/runtime_experiment.py, experiments/runtime_config.py, evaluation/metrics.py. Contract: configuration and trained models CSV and JSON artifacts, paired intervals, markdown report. It trains five agents, validates, checks fingerprint disjointness, runs the held-out test, computes paired crossed-bootstrap intervals and writes every artifact.

4. Workflow diagram for each major module

Each module has a Mermaid flowchart. Diamonds are decision points, and the numbers in each diagram match the numbered steps.

M1. Workload generator and dataset builder

Source: workload/generator.py, workload/models.py

Diagram 3 — Workflow of M1, Workload generator and dataset builder

```mermaid
flowchart TD
    N1["1. Select family configuration"]
    N2["2. Derive the workload seed"]
    N3["3. Sample 15 processes"]
    N4{"4. Valid burst, arrival and priority values?"}
    N5["5. Fingerprint and tag the workload"]
    N6["6. Assemble the split"]
    X1(["Reject workload: ValidationError"])
    N1 --> N2
    N2 --> N3
    N3 --> N4
    N4 -->|"yes"| N5
    N4 -->|"no"| X1
    N5 --> N6
```

Each workload is drawn from its family rules with a SHA-256 derived seed, validated, and fingerprinted before it is assigned to a split.

M2. Discrete-event runtime environment

Source: scheduler/runtime.py, one pass per decision epoch

Diagram 4 — Workflow of M2, Discrete-event runtime environment

```mermaid
flowchart TD
    N1["1. Advance to the next decision epoch"]
    N2["2. Admit arrivals up to the epoch"]
    N3["3. Freeze the causal observation"]
    N4["4. Obtain the policy from the controller"]
    N5{"5. Selected PID differs from the running PID?"}
    N6["6. Charge one switch-cost unit"]
    N7["7. Execute the segment"]
    N8["8. Settle the endpoint"]
    N9["9. Return interval reward r = -dW / q"]
    N10{"10. All processes completed?"}
    X1(["Emit schedule trace and six metrics"])
    N1 --> N2
    N2 --> N3
    N3 --> N4
    N4 --> N5
    N5 -->|"yes"| N6
    N5 -->|"no, or first dispatch"| N7
    N6 --> N7
    N7 --> N8
    N8 --> N9
    N9 --> N10
    N10 -->|"no"| N1
    N10 -->|"yes"| X1
```

The CPU advances from one decision epoch to the next, charges the switch cost whenever the running process changes, and returns the interval reward. Numbered steps match the diagram.

M3. Causal observation and state encoder

Source: rl/runtime_state.py

Diagram 5 — Workflow of M3, Causal observation and state encoder

```mermaid
flowchart TD
    N1["1. Receive the RuntimeObservation"]
    N2["2. Extract five causal features"]
    N3["3. Discretise each feature relative to the quantum"]
    N4["4. Flatten the bins in mixed-radix order"]
    N5["5. Return state index s in [0, 162)"]
    N1 --> N2
    N2 --> N3
    N3 --> N4
    N4 --> N5
```

The frozen observation becomes one of 162 state indices. Any object that is not a RuntimeObservation, and an empty ready queue, is rejected.

M4. Sequential Q-learning controller

Source: rl/runtime_controller.py, rl/q_learning.py

Diagram 6 — Workflow of M4, Sequential Q-learning controller

```mermaid
flowchart TD
    N1["1. Encode the observation: s = encode(obs)"]
    N2{"2. Training run?"}
    N3{"3. First visit to s, or random draw below epsilon?"}
    N4["4. Choose a uniformly random action"]
    N5["5. Choose the greedy action over visited actions"]
    N6{"6. Any visited action in s?"}
    N7["7. Count an unseen-state fallback to the heuristic"]
    N8["8. Choose the greedy action over visited actions"]
    N9["9. Receive reward r = -dW / q"]
    N10["10. Update Q(s,a) from visited next-state actions"]
    N11["11. Persist Q values and visit counts"]
    X1(["Return action (evaluation, read-only)"])
    N1 --> N2
    N2 -->|"yes"| N3
    N2 -->|"no: evaluation"| N6
    N3 -->|"yes"| N4
    N3 -->|"no"| N5
    N6 -->|"yes"| N8
    N6 -->|"no"| N7
    N7 --> X1
    N8 --> X1
    N4 --> N9
    N5 --> N9
    N9 --> N10
    N10 --> N11
```

Training chooses epsilon-greedily; evaluation is greedy and read-only. Each reward updates Q from visited next-state actions, and unseen states are counted as fallbacks.

M5. Predeclared heuristic controller

Source: rl/runtime_heuristic.py (fixed rules, no learning)

Diagram 7 — Workflow of M5, Predeclared heuristic controller

```mermaid
flowchart TD
    N1["1. Receive the same frozen observation"]
    N2{"2. At least 3 ready jobs and mean arrival age >= quantum?"}
    N3{"3. At least 2 ready jobs with distinct priorities?"}
    N4{"4. Largest remaining burst >= 2 x smallest?"}
    X1(["Return Round Robin"])
    X2(["Return Priority"])
    X3(["Return SJF"])
    X4(["Return FCFS (default)"])
    N1 --> N2
    N2 -->|"yes"| X1
    N2 -->|"no"| N3
    N3 -->|"yes"| X2
    N3 -->|"no"| N4
    N4 -->|"yes"| X3
    N4 -->|"no"| X4
```

The same frozen observation is tested against three fixed rules in order; the default is FCFS.

M6. Experiment orchestration, metrics and reporting

Source: experiments/runtime_experiment.py, evaluation/metrics.py

Diagram 8 — Workflow of M6, Experiment orchestration, metrics and reporting

```mermaid
flowchart TD
    N1["1. Train five agents on six families"]
    N2["2. Validate on 140 workloads"]
    N3{"3. Training and validation fingerprints disjoint?"}
    N4["4. Select the illustrative trace by the validation-only rule"]
    N5["5. Generate the 210 held-out test workloads"]
    N6{"6. Validation and test fingerprints disjoint, and training disjoint from both?"}
    N7["7. Run the held-out test and derive six metrics per workload"]
    N8["8. Compare pairwise: paired differences and crossed bootstrap"]
    N9["9. Write CSV, JSON, Q-table, coverage and report artifacts"]
    X1(["Stop: ValidationError"])
    N1 --> N2
    N2 --> N3
    N3 -->|"yes"| N4
    N3 -->|"no"| X1
    N4 --> N5
    N5 --> N6
    N6 -->|"yes"| N7
    N6 -->|"no"| X1
    N7 --> N8
    N8 --> N9
```

Training, validation, two disjointness checks, the held-out test, paired statistics and artifact export run in this order, and any failed check stops the run.

5. Dataset details and data collection procedure

5.1 Data collection procedure

No external or personal data is used. The dataset is synthetic and generated entirely by the project's own generator, so it can be reproduced from the declared integers on any machine.

- **D1:** Fix seven family configurations — burst distribution, arrival pattern and priority pattern, declared once in config.py and frozen in the runtime experiment configuration.
- **D2:** Derive one seed per workload — workload seed = SHA-256(master seed, split tag, family index, repetition or episode); no global random state is shared.
- **D3:** Generate and validate 15 processes per workload — burst, arrival and priority are drawn by the family rules, then checked for positive burst, non-negative arrival and bounded priority.
- **D4:** Assign workloads to splits — training: 1,200 sequential episodes for each of five seeds (6,000 in total), with the six training families cycled so each receives 200 episodes per seed; validation: 140 workloads (7 families x 20); held-out test: 210 workloads (7 families x 30).
- **D5:** Fingerprint and check disjointness — each workload has a SHA-256 fingerprint; the pipeline asserts zero overlap across the splits before the held-out test runs.
- **D6:** Store what is needed and regenerate on demand — metrics, the manifest, Q tables and summaries are committed; the large per-decision logs are regenerated by python main.py runtime-experiment and are gitignored.

5.2 Family specification

Each family generates 15 processes. Parameters are read from the frozen configuration and are also stored in results/runtime/runtime_summary.json.

- **short_jobs —** 80% of bursts uniform over 1–5 and the rest uniform over 6–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5. Role: training and evaluation.
- **long_jobs —** 20% of bursts uniform over 1–5 and the rest uniform over 6–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5. Role: training and evaluation.
- **mixed —** Bursts uniform over 1–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5. Role: training and evaluation.
- **cpu_bursty —** Bursts uniform over 20–50; arrival times uniform over the integers 0–15; priorities uniform over 1–5. Role: training and evaluation.

- **priority_skewed —** Bursts uniform over 1–50; arrival times uniform over the integers 0–20; priorities uniform over 1–5, except that with probability 0.7 the priority is drawn from 1–2 instead. Role: training and evaluation.
- **staggered_interactive —** One long process with burst uniform over 25–50 at time 0, and 14 short processes with bursts uniform over 1–3; first arrival at time 0, then integer gaps uniform over 4–6; all priorities equal to 1. Role: training and evaluation.
- **poisson_arrivals —** Bursts uniform over 1–20; arrivals from a Poisson process with rate 0.5 (exponential inter-arrival times, floored to whole time units), first arrival at time 0; priorities uniform over 1–5. Role: evaluation only; never used for training.

5.3 Measured dataset statistics

Recomputed with the project's generator at build time, so they describe the held-out dataset rather than the configuration alone. High priority means priority <= 2.

- **All families —** 210 workloads, 3,150 processes; burst 18.8 ± 15.8; mean arrival 13.7; mean total work 282; high-priority share 55%.

5.4 Data integrity and assumptions

- Reproducibility and disjointness — a workload is a pure function of its seed inputs, and the committed run reports overlap = false across 6,000 distinct training fingerprints.
- Burst knowledge — exact bursts are known once a process arrives, as standalone SJF already requires; no burst predictor is included.
- Licensing and privacy — the dataset is entirely synthetic, so no personal or licensed data is involved.

6. Tools, technologies, algorithms and frameworks used

6.1 Tools and technologies

- **Language:** Python 3.11.2 — implementation, command-line interface and report builders.
- **Numerics and data:** NumPy 2.4.6, pandas 3.0.6 — Q tables and visit counts, bootstrap resampling, metric aggregation and CSV export.
- **Testing:** pytest 9.1.1 — automated checks of every module (section 8.2).
- **Diagrams:** Mermaid (text source) — all diagrams in this document.
- **Document generation:** python-docx 1.2.0, ReportLab 5.0.1 — the text-only DOCX and the PDF rendered from it.
- **Standard library:** hashlib (SHA-256), argparse, dataclasses, json, pathlib — seed derivation, fingerprints, configuration and serialisation.
- **Version control:** Git and GitHub — branch-per-change workflow with pull requests.

No deep-learning framework is used. The controller is a tabular Q-learning agent, so every value, visit count and fallback is inspectable in the committed artifacts.

6.2 Algorithms

- **FCFS (action 0) —** dispatch the head of the ready queue, non-preemptively. Fixed baseline and runtime action.
- **SJF (action 1) —** smallest remaining burst first, non-preemptively. Minimises mean waiting time when all jobs are present together; a strong but not guaranteed-optimal reference under staggered arrivals.
- **Round Robin (action 2) —** the head runs for min(quantum, remaining burst); unfinished jobs go to the tail. Bounds the time to first service.
- **Priority (action 3) —** highest static priority first, non-preemptively. Fixed baseline and runtime action; a lower numeric value means higher priority.
- **Tabular Q-learning —** epsilon-greedy exploration, unvisited-action masking, the update of section 1.4. The learned runtime controller over 162 states x 4 actions.
- **Predeclared heuristic —** Round Robin, then Priority, then SJF, then FCFS, on the same observation. The non-learning adaptive reference.
- **Crossed bootstrap —** resample workloads within each family and model seeds independently; 1,000 replicates. Paired 95% intervals.

6.3 Controller pseudocode

```
SELECT_ACTION(obs):                 # training: epsilon-greedy; evaluation: greedy
    s <- encode(obs)                # 5 causal features -> state in [0, 162)
    if evaluating:
        if no visited action in s:  # wholly unseen state
            fallback_count += 1
            return HEURISTIC(obs)   # predeclared rule, reported rather than hidden
        return argmax over visited actions of Q[s, a]
    if first_visit(s) or rand() < epsilon(episode):
        return uniform_random_action()
    return argmax over visited actions of Q[s, a]

UPDATE(s, a, r, s', terminal):
    target <- r if terminal else r + gamma * max over visited actions of Q[s', .]
    Q[s, a] <- Q[s, a] + alpha * (target - Q[s, a])
    visits[s, a] <- visits[s, a] + 1

HEURISTIC(obs):                     # predeclared, no learning
    if ready_count >= 3 and mean_arrival_age >= quantum:  return Round Robin
    if at least 2 ready jobs have distinct priorities:    return Priority
    if max(remaining_burst) >= 2 * min(remaining_burst):  return SJF
    return FCFS
```

7. Experimental plan and evaluation metrics

7.1 Experimental plan

- **P1:** Fix the design before looking at results — families, splits, seeds, quantum (4), switch cost (1), learning rate and the heuristic rule are declared in configuration and committed.
- **P2:** Train and validate — five independent agents (seeds 7101–7105), 1,200 sequential episodes each; 140 validation workloads (seed 8201), used for reporting and the illustrative-trace rule only.
- **P3:** Test once, held out — 210 untouched workloads (seed 19301); the fixed policies and the heuristic run once; the five Q-learning models are evaluated greedily and read-only.
- **P4:** Compare and quantify uncertainty — paired per-workload differences; family-stratified crossed bootstrap, 1,000 replicates, seed 104729; 95% intervals.

7.2 Evaluation metrics

- **Mean waiting time —** mean of (turnaround time minus burst time) over processes. Why: the primary metric; the quantity the reward accumulates.
- **Mean turnaround time —** mean of (completion time minus arrival time). Why: end-to-end latency.
- **Mean response time —** mean of (first dispatch time minus arrival time). Why: interactivity; the metric Round Robin improves.
- **CPU utilisation and throughput —** busy time over elapsed time (%); processes completed per unit time. Why: the cost of switching and idling; work completed.
- **Context and policy switches —** changes of the running PID; decisions that change the policy. Why: switching overhead, and how much the controller adapts.

7.3 Results on the held-out test

Means ± standard deviation across the 210 held-out workloads, with quantum 4 and switch cost 1. Q-learning also averages the five model seeds.

- **FCFS —** waiting 127.797 ± 79.81; turnaround 146.612 ± 90.51; response 127.797 ± 79.81; CPU utilisation 92.24 ± 6.15 %; throughput 0.077 ± 0.056; context switches 14.00 ± 0.00; policy switches per trace 0.00.
- **SJF —** waiting 93.345 ± 68.78; turnaround 112.161 ± 79.39; response 93.345 ± 68.78; CPU utilisation 92.24 ± 6.15 %; throughput 0.077 ± 0.056; context switches 14.00 ± 0.00; policy switches per trace 0.00.
- **Round Robin —** waiting 215.729 ± 172.82; turnaround 234.544 ± 183.45; response 26.220 ± 9.15; CPU utilisation 78.22 ± 1.90 %; throughput 0.068 ± 0.054; context switches 74.77 ± 40.70; policy switches per trace 0.00.
- **Priority —** waiting 129.207 ± 80.81; turnaround 148.022 ± 91.53; response 129.207 ± 80.81; CPU utilisation 92.24 ± 6.15 %; throughput 0.077 ± 0.056; context switches 14.00 ± 0.00; policy switches per

trace 0.00.

- **Causal heuristic —** waiting 203.404 ± 159.23; turnaround 222.220 ± 169.85; response 40.070 ± 17.05; CPU utilisation 80.48 ± 2.22 %; throughput 0.071 ± 0.058; context switches 67.37 ± 39.49; policy switches per trace 3.21.
- **Runtime Q-learning —** waiting 104.789 ± 71.43; turnaround 123.604 ± 81.86; response 86.341 ± 59.82; CPU utilisation 90.84 ± 7.04 %; throughput 0.076 ± 0.054; context switches 17.32 ± 4.66; policy switches per trace 4.73.

7.4 Paired comparisons

Each line gives target minus reference with its 95% crossed-bootstrap interval (waiting and response time; all metrics are in results/runtime/runtime_paired_comparisons.csv). Turnaround differences equal waiting differences, because both methods serve the same bursts.

- **Runtime Q minus SJF —** waiting +11.443 [+5.022, +19.297], favours the reference; response -7.004 [-9.541, -4.305], favours the target
- **Runtime Q minus Heuristic —** waiting -98.616 [-106.097, -90.192], favours the target; response +46.272 [+43.472, +49.100], favours the reference
- **RR minus SJF —** waiting +122.383 [+117.619, +127.315], favours the reference; response -67.126 [-70.219, -64.285], favours the target
- **Heuristic minus SJF —** waiting +110.059 [+106.219, +114.032], favours the reference; response -53.276 [-56.026, -50.873], favours the target
- **FCFS minus SJF —** waiting +34.451 [+32.640, +36.540], favours the reference; response +34.451 [+32.605, +36.440], favours the reference
- **Priority minus SJF —** waiting +35.862 [+33.987, +37.684], favours the reference; response +35.862 [+33.956, +37.823], favours the reference

7.5 What the plan has established

- Against Round Robin, mean waiting time is lower by 110.940 time units (104.789 against 215.729). No paired interval was computed for this pair; it follows from the two committed comparisons against SJF (Q minus SJF +11.443; RR minus SJF +122.383). FCFS and Priority lie between them, at +34.451 and +35.862 relative to SJF.
- Against the predeclared adaptive rule, mean waiting time is lower by 98.616 (paired difference -98.616; 95% interval [-106.097, -90.192]). The rule has the lower response time, by 46.272, because it selects Round Robin whenever three or more aged jobs are ready.
- Against SJF the result is mixed. Waiting time is higher for Q-learning by 11.443 (95% interval 5.022 to 19.297), while response time is lower by 7.004. Non-preemptive SJF with exact bursts and no mid-job switch cost is a strong reference, and the learned controller does not beat it on waiting time.
- Q-learning's mean waiting time lies between the SJF and Round Robin values on all 7 held-out families, including the unseen poisson_arrivals family (68.91, against SJF 49.76 and Round Robin 113.58).
- Controller overhead is small: about 13.69 microseconds to build an observation and 21.24 microseconds to select an action per decision on the build host. These timings are never charged to the schedule.

8. Implementation progress

Review 2 requires at least 50% of the proposed work to be complete, with completed modules shown by valid outputs. All six proposed modules are implemented, tested and demonstrated below (100% of the Review 2 scope). Work planned beyond that scope is listed in section 9.2.

8.1 Completion and outputs by module

- **M1 — Workload generator and dataset builder.** completed (100%). Output: 6,350 manifest rows; 210 held-out workloads (3,150 processes) regenerated from the declared seeds, with the measured burst, arrival and priority statistics of section 5.
- **M2 — Discrete-event runtime environment.** completed (100%). Output: 2,100 final-test metric rows produced from simulated traces; fixed-action runs reproduce the standalone FCFS, SJF, Round Robin and Priority schedules exactly in the test suite.

- **M3 — Causal observation and state encoder.** completed (100%). Output: 162-state encoder; 81-89 distinct states visited per agent during training.
- **M4 — Sequential Q-learning controller.** completed (100%). Output: 3,240 Q values stored; 267-276 of 648 state-action pairs visited; 1-3 counted unseen-state fallbacks per model on the held-out test.
- **M5 — Predeclared heuristic controller.** completed (100%). Output: heuristic rows present for all 210 held-out workloads; mean waiting 203.404 with 3.21 policy switches per trace.
- **M6 — Experiment orchestration, metrics and reporting.** completed (100%). Output: 36 paired comparisons with 95% bootstrap intervals, the coverage table, the learned-switch demo trace and the generated runtime_report.md.

Overall completion of the proposed Review 2 modules: 6 of 6 (100%).

8.2 Demonstration 1 — automated verification

The suite collects 273 checks across 16 test files. The two legacy files exercise the earlier offline selector, which is kept for comparison.

Output of python -m pytest -q:

```
$ python -m pytest -q
273 passed in 12.59s
```

8.3 Demonstration 2 — end-to-end run output

One command runs the full pipeline. The output below reproduces its printed lines, with file paths shortened to file names. A fresh run reproduced the committed artifacts except for host-timing columns.

Output of python main.py runtime-experiment:

```
$ python main.py runtime-experiment
Causal event-driven runtime experiment
--------------------------------------
  training seeds                : 7101, 7102, 7103, 7104, 7105
  training episodes/model       : 1200
  validation workloads          : 140
  final held-out test workloads  : 210
  final-test mean waiting time:
    Causal heuristic      : 203.404
    FCFS                  : 127.797
    Priority              : 129.207
    Round Robin           : 215.729
    Runtime Q-learning    : 104.789
    SJF                   : 93.345
  learned within-trace switches : 15 (23 decisions; runtime_learned_switch_demo.json)
  artifacts:
    training_metrics        : runtime_training_metrics.csv
    validation_metrics      : runtime_validation_metrics.csv
    final_test_metrics      : runtime_final_test_metrics.csv
    validation_decisions    : runtime_validation_decisions.csv
    final_test_decisions    : runtime_final_test_decisions.csv
    workload_manifest       : runtime_workload_manifest.csv
    state_action_coverage   : runtime_state_action_coverage.csv
    q_table                 : runtime_q_table.csv
    paired_comparisons      : runtime_paired_comparisons.csv
    final_test_summary      : runtime_final_test_summary.csv
    validation_summary      : runtime_validation_summary.csv
    summary                 : runtime_summary.json
    learned_switch_demo     : runtime_learned_switch_demo.json
    report                  : runtime_report.md
```

8.4 Demonstration 3 — learned behaviour within one trace

The controller switches policies inside a single trace. The example, chosen by a validation-only rule (maximise policy switches among traces with zero unseen-state fallbacks), is training seed 7103, family staggered_interactive, repetition 0, fingerprint 275409d7458331f6. It contains 15 policy switches over 23 decisions, with mean waiting time 5.933 and mean response time 2.933. It illustrates the mechanism and is not a performance claim.

8.5 Artifacts produced

The run writes results/runtime/: final-test metrics (2,100 rows), validation metrics (1,400 rows), training metrics (6,000 rows), paired comparisons (36 rows), state-action coverage (15 rows), the Q table (3,240 rows), the workload manifest (6,350 rows), the illustrative trace, a summary JSON and a markdown report.

9. Challenges encountered and plan for completing the remaining work

9.1 Challenges encountered

- **Keeping the controller causal —** Why it mattered: an earlier whole-workload selector saw future arrivals, so its results could not be read as a runtime controller. Resolution: an immutable RuntimeObservation, an encoder that accepts only that object, and a hidden-future test.
- **FCFS after a Round-Robin requeue —** Why it mattered: the requeued job could be ordered by original arrival or by live queue order. Resolution: one persistent FIFO ready queue with canonical (arrival time, pid) insertion; fixed-action runs are tested against the standalone schedulers.
- **Charging switch cost fairly —** Why it mattered: charging only the learner would invalidate the comparison; charging nothing would be unrealistic. Resolution: one unit whenever the running PID changes, for every method including the fixed policies.
- **A reward that matches the objective —** Why it mattered: a shaped reward can drift from the reported metric. Resolution: r = -dW / q with gamma = 1, so the episodic return equals minus total waiting time divided by q; a test asserts this.
- **Sparse coverage of the tabular space —** Why it mattered: only about 42% of the 648 state-action pairs are visited, and unseen states appear at evaluation. Resolution: forced exploration on first visit, masking of unvisited actions, and a counted fallback to the heuristic.
- **Reporting a negative result —** Why it mattered: the learner does not beat non-preemptive SJF on mean waiting time. Resolution: reported as a finding with its interval, and the structural reason stated: SJF uses exact bursts and pays no mid-job switch cost.
- **Host-dependent timing —** Why it mattered: wall-clock columns differ between machines and can look like non-determinism. Resolution: timings are stored separately, never charged to simulated time, and excluded from schedule comparisons.

9.2 Plan for completing the remaining work

- **W1 — Sensitivity study.** gap: all results use quantum 4 and switch cost 1. Approach: re-run the plan over a grid of quanta and switch costs. Deliverable: metric tables for each cell.
- **W2 — Burst prediction.** gap: exact bursts are assumed known at arrival. Approach: add an explicit predictor and re-run the controller under its error. Deliverable: results with error bands.
- **W3 — Richer state abstraction.** gap: only about 42% of pairs are visited; 162 states is coarse. Approach: tile coding or a small function approximator, keeping the same causal contract. Deliverable: coverage and metric comparison.
- **W4 — Multi-objective reward.** gap: waiting time is optimised; response time is not. Approach: add a weighted or constrained objective. Deliverable: trade-off front over the two metrics.
- **W5 — I/O and blocking.** gap: processes never block; the queue is CPU-bound only. Approach: model I/O bursts and blocked and wake-up transitions. Deliverable: re-baselined comparisons.
- **W6 — Multicore extension.** gap: single CPU only. Approach: N cores with per-core queues and a migration cost. Deliverable: multicore metrics and overhead.
- **W7 — Trace-driven validation.** gap: all workloads are synthetic. Approach: replay a public scheduling trace through the same environment. Deliverable: trace-driven result table.
- **W8 — Phase-aware monitored features (proposed, not implemented).** gap: the encoder monitors queue size, completed count, median remaining burst, mean arrival age and priority spread; recent burst history, accumulated ready-queue waiting time and arrival rate are not yet monitored. Approach: add these as causal features, detect workload phases, and let the controller switch at phase boundaries. Deliverable: a re-trained controller compared with the current 162-state version.

9.3 Sequence for the remaining work

W1 and W2 come first, because they test whether the conclusions depend on fixed parameters or on the idealised burst assumption. W8 and W3 follow, since they address the coarse state abstraction and the missing monitored features. W4 to W6 extend the environment, and W7 provides external validation.